PIRIS Mathieu

Dossier pédagogique E6 - conception et développement d’application (option SLAM)

Brevet de Technicien Supérieur - Services Informatiques aux Organisations

Session 2026

Épreuve E6 - Conception et développement d'applications (option SLAM)

ANNEXE 7-1-B : Fiche descriptive de réalisation professionnelle (recto)

DESCRIPTION D'UNE RÉALISATION PROFESSIONNELLE

Nom, prénom : PIRIS Mathieu	N° candidat : 2542561704
Épreuve ponctuelle X ☐ Contrôle en cours de formation ☐	N° réalisation : 1

Organisation support de la réalisation professionnelle

Intitulé de la réalisation professionnelle
Urthel - Boutique de vente de consoles de jeux vidéo

Cette application propose une boutique en ligne dédiée à la vente de consoles de jeux vidéo neuves et d'occasion. Elle permet aux visiteurs de consulter le catalogue de consoles, filtré par catégorie et état, et aux utilisateurs enregistrés d'ajouter des articles à leur panier. L'administrateur dispose d'un panneau de gestion complet permettant de créer, modifier et supprimer des consoles, ainsi que de gérer les catégories et les utilisateurs.

Période de réalisation : 2025 - 2026 Lieu : Stage et centre de formation
Modalité : X Seul(e) ☐ En équipe

Compétences travaillées
X Concevoir et développer une solution applicative
X Assurer la maintenance corrective ou évolutive d'une solution applicative
X Gérer les données

Conditions de réalisation (ressources fournies, résultats attendus)

Dans le cadre de ma formation BTS SIO option SLAM, j'ai été chargé de concevoir et développer une application web complète de boutique e-commerce spécialisée dans la vente de consoles de jeux vidéo. L'objectif était de mettre en place une solution permettant :

1. **La consultation d'un catalogue de consoles avec filtres par catégorie et état**
2. **Un système de panier d'achat pour les utilisateurs connectés**
3. **Une interface d'administration pour la gestion CRUD des consoles**
4. **Un système de gestion des catégories (PlayStation, Xbox, Nintendo, Rétro, Portable)**
5. **Un mode sombre / clair via un bouton de bascule**
6. **Une interface responsive et moderne**

Ressources Fournies

- **Documentation Laravel 12**
- **Environnement de développement XAMPP (Windows 11)**
- **Base de données MySQL 8.0**
- **Accès Internet pour documentation**

Résultats Attendus

- **Application web fonctionnelle et sécurisée**
- **Système de rôles et permissions opérationnel**
- **Interface utilisateur responsive avec dark mode**
- **CRUD complet pour la gestion des consoles**

Description des ressources documentaires, matérielles et logicielles utilisées

Matérielles

- **Poste de développement :** Windows 11 (64 bits)
  - 8 Go RAM, 4 CPU
  - XAMPP 8.2 (Apache + MySQL + PHP)
- **VirtualBox :** Configuration réseau NAT (utilisé pour tests en conditions serveur)

Logicielles

Backend :

- PHP 8.2.12
- Laravel 12.x
- Composer 2.x
- MySQL 8.0

Serveur local :

- Apache 2.4 (via XAMPP)
- phpMyAdmin 5.x
- GLPI (gestion des incidents)

Frontend :

- Blade (moteur de templates Laravel)
- Bootstrap 5.3.0 (via CDN)
- Bootstrap Icons 1.11.0
- JavaScript ES6 (dark mode toggle)
- Vite (compilation assets SCSS)

Outils de développement :

- Visual Studio Code
- Git 2.x
- GitHub (dépôt distant)
- phpDocumentor 3.9 (génération de documentation)

Bibliothèques :

- Spatie Laravel Permission 6.24 (gestion des rôles)
- Laravel UI 4.6 (authentification)
- PHPUnit 11.5 (tests)

Documentaires

- Documentation officielle Laravel : https://laravel.com/docs/12.x
- Documentation Spatie Permission : https://spatie.be/docs/laravel-permission
- Stack Overflow et communauté Laravel

- Documentation MySQL et Apache
- Documentation Bootstrap 5.3 : https://getbootstrap.com/docs/5.3

Modalités d'accès aux productions et à leur documentation	**Comptes de test :** **Admin :** - Email : admin@admin.fr - Password : password **Utilisateur :** - Créer via le formulaire d'inscription **Accès phpMyAdmin :** URL : http://localhost/phpmyadmin User : root Password : (vide)
URL Développement : http://urthel.local Github : https://github.com/Valaraukar56/projet-console-laravel Documentation phpDoc : accessible depuis le footer admin	

BTS SERVICES INFORMATIQUES AUX ORGANISATIONS **SESSION 2026**

Épreuve E6 - Conception et développement d'applications (option SLAM)

Descriptif de la réalisation professionnelle, y compris les productions réalisées et schémas explicatifs

1. CONTEXTE ET OBJECTIFS

1.1 Contexte du projet

Le projet **Urthel** a été développé dans le cadre de ma formation BTS SIO option SLAM à l'AFTEC Vannes. Il répond à un besoin identifié de vente en ligne de consoles de jeux vidéo neuves et d'occasion permettant :

- Présenter les consoles de manière attractive avec images et états détaillés
- Permettre aux clients de consulter les produits, leurs prix et leurs états
- Offrir un système de panier pour préparer les commandes
- Gérer facilement le catalogue via une interface d'administration
- Proposer un mode sombre pour améliorer le confort de navigation

L'application propose une triple approche :

1. Une vitrine publique accessible à tous les visiteurs
2. Un espace client avec panier pour les utilisateurs enregistrés
3. Un panneau d'administration pour la gestion des consoles

1.2 Objectifs techniques détaillés

Objectif principal

Concevoir et développer une application web full-stack avec le framework Laravel 12, déployée localement via XAMPP.

Objectifs spécifiques

1. Architecture Client-Serveur

- Mettre en place une architecture 3-tiers (présentation, logique métier, données)
- Configurer l'environnement de développement avec XAMPP
- Assurer la communication HTTP entre client et serveur

2. Système CRUD Complet

- Implémenter les opérations Create, Read, Update, Delete pour les consoles
- Gérer les relations entre les entités (utilisateurs, consoles, catégories, panier)
- Valider les données côté serveur avec les validations Laravel

3. Authentification Sécurisée

- Utiliser Laravel UI pour le système d'authentification
- Implémenter le hashage des mots de passe avec Bcrypt
- Protéger les routes avec les middlewares d'authentification

4. Système de Rôles et Permissions

- Intégrer Spatie Laravel Permission pour la gestion des rôles
- Définir deux rôles principaux : admin et user
- Restreindre l'accès aux fonctions d'administration

5. Expérience Utilisateur

- Implémenter un dark mode via Bootstrap 5.3 data-bs-theme
- Assurer un design responsive avec Bootstrap 5.3
- Permettre le filtrage des consoles par catégorie et état

2. ANALYSE ET CONCEPTION

2.1Analysedesbesoinsfonctionnels

Besoins identifiés

Pour les visiteurs (non authentifiés) :

- Consulter le catalogue de consoles
- Filtrer par catégorie (PlayStation, Xbox, Nintendo, Rétro, Portable)
- Filtrer par état (neuf, très bon, bon, acceptable)
- Voir les détails d'une console (nom, description, prix, état, image)

Pour les utilisateurs enregistrés :

- Toutes les fonctionnalités des visiteurs
- Ajouter des consoles au panier
- Modifier les quantités dans le panier
- Supprimer des articles du panier
- Voir le total du panier

Pour les administrateurs :

- Toutes les fonctionnalités des utilisateurs
- Créer de nouvelles consoles avec upload d'image
- Modifier les consoles existantes
- Supprimer des consoles du catalogue
- Gérer la disponibilité des consoles
- Accéder à GLPI, phpMyAdmin et la documentation phpDoc

2.2 Cas d'utilisation principaux

UC1 : Consulterlecataloguedeconsoles

Acteur : Visiteur / Utilisateur

Précondition : Aucune

Scénario nominal :

1. Le visiteur accède à la page d'accueil
2. Il visualise la grille des consoles avec images, noms, états et prix
3. Il peut filtrer par catégorie ou état via les listes déroulantes
4. Il peut cliquer sur une console pour voir ses détails

UC2 : Gérer son panier

Acteur : Utilisateur authentifié

Précondition : L'utilisateur est connecté

Scénario nominal :

1. L'utilisateur clique sur "Ajouter au panier"
2. La console est ajoutée avec une quantité de 1 (incrémentée si déjà présente)
3. L'utilisateur accède à son panier

4. Il peut modifier les quantités ou supprimer des articles
5. Le total est calculé automatiquement

UC3 : Administrer les consoles

Acteur : Administrateur
Précondition : L'utilisateur a le rôle admin
Scénario nominal :

1. L'administrateur clique sur "Ajouter une console"
2. Il remplit le formulaire (nom, description, prix, état, catégorie, image)
3. La console est créée et visible dans le catalogue

2.3 Diagramme de classes

```
+------------------+      +------------------+      +------------------+      + ------------------+
|      User        |      |    Category      |      |     Console      |      | CartItem | + ------
+------------------+      +------------------+      +------------------+      ------------+  |  id:
| id: bigint       |      | id: bigint       |      | id: bigint       |      bigint  |  |  user_id:
| name: string     |      | name: string     |      | name: string     |      FK | | console_id: FK
| email: string    |      | slug: string     |      | description:txt|        | | quantity: int | |
| password: string |      | description: txt|       | price: decimal          created_at     |     |
||   created_at          || image: string          | image: string           updated_at | + ------
|updated_at +---------    || created_at             | condition:enum          ------------+
---------+                || updated_at             |
        |                +------------------+      | category_id:  FK
        |                        |1                |
        |                        |                 is_available:bool|   |
        |1                       |                 |    created_at     ||
        |                        |N                u+p d-a-t-e-d-_-a-t-----------+
        |N                 + ------- +
        + ------ +
        |

           hasMany / belongsTo
```

Relations :

- User hasMany CartItem (un utilisateur peut avoir plusieurs articles dans son panier)
- CartItem belongsTo User (un article appartient à un utilisateur)
- CartItem belongsTo Console (un article référence une console)
- Console hasMany CartItem (une console peut être dans plusieurs paniers)
- Category hasMany Console (une catégorie regroupe plusieurs consoles)
- Console belongsTo Category (une console appartient à une catégorie)

2.4 Diagramme d'entité-association

```
+ ------------------+                    + ------------------+
| (T) User | + ------                    |(T)Category | + ----
------------+ |  id  :                   --------------+
BIGINT  |  |  name  :                    |id:BIGINT          |
VARCHAR |  |  email  :                   |name:VARCHAR       |
VARCHAR |                          1..N  |slug:VARCHAR |
| password:VARCHAR |               +-----| description:TEXT |
| created_at : TS  |               |     | image : VARCHAR  |
| updated_at : TS  |               |     | created_at : TS  |
+ ------------------+               |     | updated_at : TS  |
        | 1                        |     + ---------------- +
        |                          |
        | possède                  | est dans
        ¦                          ¦
        | N                        | N
+ ---------------------------------- ■
|         (T) CartItem              |
+ ---------------------------------- ■
| id : BIGINT                       |
```

```
        | user_id : BIGINT (FK) |
      | console_id : BIGINT (FK) |
         | quantity : INTEGER |                  + ------------------+
       | created_at : TIMESTAMP |                | (T) Console | + ---
       | updated_at : TIMESTAMP |                ---------------+ | id
+ ----------------------------------+            : BIGINT  |  | name  :
                  |                             VARCHAR |
                                      1..N
                 +-------------------------+---| description: TEXT|
                                           | | price: DECIMAL   |
                                           | | image: VARCHAR   |
                                           | | condition: ENUM  |
                                           | | category_id: FK  |
                                           | | is_available:BOOL|
                                           | | created_at: TS   |
                                           | | updated_at: TS   |
                                           | + ------------------+
                                           |
                                     Table d'association entre
                                     User et Console (N:N avec
                                     attribut quantity)
```

2.5 Architecture MVC

```
■ ---------------------------------------------------------------------------■
|                         NAVIGATEUR                                         |
|                     (Client HTTP/HTML)                                     |
■ ---------------------------------------------------------------------------■
                              |
                              ■
■ ---------------------------------------------------------------------------■
|                       SERVEUR APACHE                                       |
|                    (XAMPP - localhost:80)                                  |
■ ---------------------------------------------------------------------------■
                              |
                              ■
■ ---------------------------------------------------------------------------■
| | | | (web.php) --> A(PCPoLnIsCoAlTeICOoNn tLrAoRlAlVeErL) 1-2- > (Blade)|
                                                                             |
   Routes           Controllers             Views                            |
                                                                            |
|                   (CartController)                                         |
|                   (HomeController)                                         |
|       |                 |                                                  |
|       v                 v                                                  |
|  Middlewares        Modèles                                                |
|  (auth, role)       (Eloquent                                              |
|                     ORM) Console, User,                                    |
|                     CartItem, Category                                     |
|                                                                            |

■ ---------------------------------------------------------------------------■
                              |
                              ■
■ ---------------------------------------------------------------------------■
|                       BASE DE DONNÉES                                      |
|                    MySQL 8.0 (projet_laravel)                              |
■ ---------------------------------------------------------------------------■
```

3. DÉVELOPPEMENT

3.1 Extrait du ConsoleController.php

```php
<?php
namespace App\Http\Controllers;

use App\Models\Console;
use App\Models\Category;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Storage;

class ConsoleController extends Controller
{

    // Liste des consoles avec filtres catégorie et état
    public function index(Request $request)
    {
        $query = Console::with('category')->where('is_available', true);

         if ($request->has('category') && $request->category){
             $query->where('category_id', $request->category);
        }
        if ($request->has('condition') && $request->condition){
            $query->where('condition', $request->condition);
        }

        $consoles = $query->latest()->get();
        $categories = Category::all();

        return view('consoles.index', compact('consoles', 'categories'));
    }

    // Création d'une console
    public function store(Request $request)
    {
        $validated = $request->validate([
            'name'        => 'required|string|min:2|max:255',
            'description' => 'nullable|string',
            'price'       => 'required|numeric|min:0.01',
            'condition'   => 'required|in:neuf,très bon,bon,acceptable',
            'category_id' => 'required|exists:categories,id',
            'image'       => 'nullable|image|max:2048',
        ]);

        if ($request->hasFile('image')) {
            $validated['image'] = $request->file('image')
                                        ->store('consoles', 'public');
        }

        Console::create($validated);

        return redirect()->route('consoles.index')
                         ->with('success', 'Console ajoutée avec succès !');
    }

    // Suppression avec nettoyage de l'image
    public function destroy(Console $console)
    {
        if ($console->image) {
            Storage::disk('public')->delete($console->image);
        }
        $console->delete();
        return redirect()->route('consoles.index')
                         ->with('success', 'Console supprimée avec succès !');
    }
}
```

3.2 Extrait des routes (web.php)

```
<?php                                   use
App\Http\Controllers\ConsoleController;  use
App\Http\Controllers\CartController;     use
App\Http\Controllers\HomeController;

// Routes publiques
Route::get('/', [ConsoleController::class, 'index'])->name('home');
Route::get('/consoles', [ConsoleController::class, 'index'])->name('consoles.index');
Route::get('/consoles/{console}', [ConsoleController::class, 'show'])->name('consoles.show');

// Routes authentifiées (panier)
Route::middleware('auth')->group(function (){
    Route::get('/cart', [CartController::class, 'index'])->name('cart.index');
    Route::post('/cart/{console}', [CartController::class, 'store'])->name('cart.store');
    Route::patch('/cart/{cartItem}', [CartController::class, 'update'])->name('cart.update');
    Route::delete('/cart/{cartItem}', [CartController::class, 'destroy'])->name('cart.destroy');
});

// Routes admin (création, modification, suppression)
Route::middleware(['auth', 'role:admin'])->group(function () {
    Route::get('/admin/consoles/create', [ConsoleController::class, 'create'])
         ->name('consoles.create');
    Route::post('/admin/consoles', [ConsoleController::class, 'store'])
         ->name('consoles.store');
    Route::get('/admin/consoles/{console}/edit', [ConsoleController::class, 'edit'])
         ->name('consoles.edit');
    Route::put('/admin/consoles/{console}', [ConsoleController::class, 'update'])
         ->name('consoles.update');
    Route::delete('/admin/consoles/{console}', [ConsoleController::class, 'destroy'])
         ->name('consoles.destroy');
});

// Routes d'authentification (Laravel UI)
Auth::routes();
```

3.3 Extrait du CartController.php

```php
<?php
namespace App\Http\Controllers;

use App\Models\CartItem;
use App\Models\Console;
use Illuminate\Support\Facades\Auth;

class CartController extends Controller
{

    public function index()
    {
        $cartItems = CartItem::with('console')
        ->where('user_id', Auth::id())
        ->get();

        $total = $cartItems->sum(fn($item) =>
            $item->console->price * $item->quantity
        );

        return view('cart.index', compact('cartItems', 'total'));
    }

    public function store(Console $console)
    {
        if (!$console->is_available){
            return redirect()->back()
                ->with('error', 'Cette console n\'est plus disponible.');
        }

        $cartItem = CartItem::where('user_id', Auth::id())
            ->where('console_id', $console->id)->first();

        $cartItem
            ? $cartItem->increment('quantity')
            : CartItem::create(['user_id' => Auth::id(),
                                'console_id' => $console->id, 'quantity' => 1]);

        return redirect()->back()->with('success', 'Console ajoutée au panier !');
    }
}
```

4. BASE DE DONNÉES

4.1Structuredesmigrations

Fichier de migration	Table créée	Description
0001_01_01_000000_create_users_table	users, sessions, password_reset_tokens	Utilisateurs et authentification
2026_01_20_131305_create_permission_tables	roles, permissions, model_has_roles...	Système Spatie Permission
2026_01_20_131322_create_categories_table	categories	Catégories de consoles
2026_01_20_131329_create_consoles_table	consoles	Catalogue de consoles
2026_01_20_131344_create_cart_items_table	cart_items	Panier utilisateur

4.2 Migration de la table consoles

```
Schema::create('consoles', function (Blueprint $table){
    $table->id();
    $table->string('name');
    $table->text('description')->nullable();
    $table->decimal('price', 8, 2);
    $table->string('image')->nullable();
    $table->enum('condition', ['neuf', 'très bon', 'bon', 'acceptable'])

          ->default('bon');
    $table->foreignId('category_id')->constrained()->onDelete('cascade');
    $table->boolean('is_available')->default(true);
    $table->timestamps();

});
```

4.3 Seeders

L'application est initialisée via 3 seeders :

- **AdminSeeder** : crée le rôle admin et l'utilisateur admin@admin.fr
- **CategorySeeder** : crée 5 catégories (PlayStation, Xbox, Nintendo, Rétro, Portable)
- **ConsoleSeeder** : insère 22 consoles réparties dans les 5 catégories

```
// AdminSeeder.php
$adminRole = Role::firstOrCreate(['name' => 'admin']);
$admin = User::firstOrCreate(
    ['email' => 'admin@admin.fr'],
    ['name' => 'Admin', 'password' => bcrypt('password')]
);
$admin->assignRole($adminRole);
```

4.4 Catégories de consoles

Catégorie	Description	Exemples de consoles
PlayStation	Consoles Sony	PS1, PS2, PS3, PS4, PS5

Xbox	Consoles Microsoft	Xbox, Xbox 360, Xbox One, Xbox Series
Nintendo	Consoles Nintendo de salon	NES, SNES, N64, GameCube, Wii, Switch
Rétro	Consoles vintage	Atari, Sega, Neo Geo
Portable	Consoles portables	Game Boy, PSP, PS Vita, 3DS

5. FONCTIONNALITÉS TECHNIQUES AVANCÉES

5.1 Système de rôles et permissions (Spatie)

L'applicationutiliselepackage**Spatie LaravelPermission v6.24** pour la gestion des rôles. Le middleware est enregistré dans bootstrap/app.php :

```
$middleware->alias([
    'role'               => \Spatie\Permission\Middleware\RoleMiddleware::class,
    'permission'         => \Spatie\Permission\Middleware\PermissionMiddleware::class,
    'role_or_permission' => \Spatie\Permission\Middleware\RoleOrPermissionMiddleware::class,
]);
```

Les vues Blade utilisent les directives @role('admin') pour afficher conditionnellement les éléments réservés aux administrateurs (bouton d'ajout, badge Admin, liens footer).

5.2 Dark Mode via Bootstrap 5.3

Bootstrap5.3supportenativementledark mode via l'attribut data-bs-theme sur la balise `<html>` .
L'implémentation comprend :

- Un script anti-flash dans le <head> pour appliquer le thème avant le chargement CSS
- Un bouton lune/soleil dans la navbar
- La persistance du choix via localStorage

```
// Anti-flash (dans <head>)
(function(){
    const t = localStorage.getItem('theme') || 'light';
    document.documentElement.setAttribute('data-bs-theme', t);
})();

// Toggle (en fin de body)
toggle.addEventListener('click', () => {
    const next = document.documentElement
                        .getAttribute('data-bs-theme') === 'dark' ? 'light' : 'dark';
    document.documentElement.setAttribute('data-bs-theme', next);
    localStorage.setItem('theme', next);
});
```

5.3 Documentation phpDoc

Ladocumentationtechnique du code est générée via **phpDocumentor 3.9** et placée dans public/docs/ , accessible directement depuis le site. Un lien est visible dans le footer uniquement pour les administrateurs.

La configuration est définie dans phpdoc.xml : source app/ , sortie public/docs/ .

5.4 Gestion des images

Lesimages deconsoles sont stockées dans storage/app/public/consoles/ via le système de disques Laravel. La suppression d'une console entraîne automatiquement la suppression de son image associée :

```
if ($console->image) {
    Storage::disk('public')->delete($console->image);
}
$console->delete();
```

5.5 Filtrage du catalogue

Lecatalogueproposeundoublefiltrage en temps réel par catégorie et par état de la console :

```
$query = Console::with('category')->where('is_available', true);

if ($request->category) {
    $query->where('category_id', $request->category);
}
if ($request->condition) {
    $query->where('condition', $request->condition);
}

$consoles = $query->latest()->get();
```

6. SÉCURITÉ ET TESTS

6.1Mesuresdesécuritéimplémentées

Menace	Mesure implémentée
Injection SQL	Eloquent ORM avec requêtes préparées, aucune requête SQL brute
CSRF	Token CSRF Laravel sur tous les formulaires POST/PUT/DELETE
XSS	Échappement automatique via le moteur Blade ({{ }})
Accès non autorisé	Middleware auth et role:admin sur les routes sensibles
Mots de passe	Hashage Bcrypt via password => 'hashed' dans les casts du modèle
Accès panier	Vérification $cartItem->user_id !== Auth::id() avant toute modification
Upload de fichiers	Validation image|max:2048 , stockage hors de la racine web

6.2 Tests réalisés

Test	Résultat
Accès à /admin/consoles/create sans être connecté	Redirection vers login ✓ Erreur 403
Accès à /admin/consoles/create avec rôle user	Forbidden ✓ Erreur 403 Forbidden
Tentative d'accès au panier d'un autre utilisateur	✓ Succès, incrémentation quantité
Ajout d'une console au panier (utilisateur connecté)	✓
Création d'une console avec image	Image stockée dans storage/app/public/consoles ✓
Suppression d'une console avec image	Fichier image supprimé du disque ✓
Filtrage par catégorie et état	Résultats corrects affichés ✓
Dark mode persistant après rechargement	Thème restauré via localStorage ✓

6.3 Versioning Git

Leprojetestversionnéavec Git et hébergé sur GitHub : `https://github.com/Valaraukar56/projet-console-laravel`

Les principales étapes de développement ont été commitées :

- Initialisation du projet Laravel et configuration XAMPP
- Création des migrations et modèles (Console, Category, CartItem)
- Implémentation du CRUD consoles avec upload d'images
- Intégration Spatie Laravel Permission (rôles admin/user)
- Développement du panier d'achat
- Ajout de la documentation phpDoc dans le footer admin
- Implémentation du dark mode Bootstrap 5.3

ANNEXE VII-7 : Modèle d'attestation de respect de l'annexe II.E – Environnement technologique pour la certification du référentiel
Épreuve E6 - Conception et développement d'applications (option SLAM)

CONTRÔLE DE L'ENVIRONNEMENT TECHNOLOGIQUE

En référence à l'annexe II.E – « Environnement technologique pour la certification » du référentiel du BTS SIO

Identification		**SLAM**

1. Environnement commun aux deux options

1.1 L'environnement technologique supportant le système d'information de l'organisation cliente comporte au moins :

Éléments	Description de l'implantation dans le centre d'examen (nom du service ou de l'outil et caractéristiques techniques)	Remarques de la commission d'interrogation
Un service d'authentification	Laravel UI avec authentification native Laravel + Spatie Laravel-Permission pour la gestion des rôles (Admin, User)	
Un SGBD	MySQL 8.0 avec phpMyAdmin pour l'administration. Base de données relationnelle avec migrations Laravel	
Un accès sécurisé à internet	Connexion HTTP pour l'accès au site. CSRF et protection des sessions Laravel	
Un environnement de travail collaboratif	GitHub pour le versioning et la collaboration (https://github.com/Valaraukar56/projet-console-laravel)	
Deux serveurs, éventuellement virtualisés, basés sur des systèmes d'exploitation différents, dont l'un est un logiciel libre (open source)	- Serveur 1 : Windows 11 + XAMPP (développement) - Serveur 2 : VirtualBox avec Ubuntu Server (tests en conditions de production)	

1.2 Des outils sont mobilisés pour la gestion de la sécurité :

Éléments		**Remarques de la commission d'interrogation**
Gestion des incidents	Logs Laravel (storage/logs) – Messages d'erreur personnalisés – GLPI pour la gestion des tickets (accessible depuis le footer admin)	
Détection et prévention des intrusions	Protection CSRF (tokens Laravel) – Middleware auth et role:admin pour bloquer les accès non autorisés – Validation des données côté serveur	
Chiffrement	Hashage des mots de passe (Bcrypt) – Chiffrement des sessions Laravel – Protection des cookies	
Analyse de trafic	Logs d'accès Apache – Logs Laravel – Suivi des requêtes SQL via Eloquent	
Une solution de sauvegarde	GitHub (versionning du code) – Export régulier de la base de données via phpMyAdmin	
Des ressources dont l'accès est sécurisé et soumis à habilitation	Accès phpMyAdmin soumis à authentification – Routes admin protégées par middleware role:admin – Documentation phpDoc accessible uniquement aux admins	
Deux types de terminaux dont un mobile	Ordinateur de bureau (développement Windows 11) + Smartphone/tablette (navigation responsive grâce à Bootstrap 5.3 et CSS media queries)	

Remarque : les logiciels de simulation ou d'émulation sont utilisés en réponse à des besoins de l'organisation. Ils ne peuvent se substituer complètement à des équipements réels dans l'environnement technologique d'apprentissage.

2. Savoirs spécifiques à l'option « solutions logicielles et applications métiers » (SLAM)

2.1 L'environnement technologique supportant le système d'information de l'organisation cliente comporte au moins :

Éléments	Description de l'implantation dans le centre d'examen (nom du service ou de l'outil et caractéristiques techniques)	Remarques de la commission d'interrogation
Un ou deux environnements de développement disposant d'outils de gestion de tests et supportant un cadre applicatif (framework) et au moins deux langages	- IDE : Visual Studio Code avec extensions PHP Intelephense, Laravel Blade Snippets - Framework : Laravel 12 - Langages : PHP 8.2, JavaScript ES6, SQL - Tests : PHPUnit 11.5 intégré à Laravel	
Une bibliothèque de composants logiciels	- Composer pour dépendances PHP - Packages : Spatie Laravel-Permission 6.24, Laravel UI 4.6 - NPM pour packages JavaScript (Bootstrap 5.3, Vite 7)	
Un SGBD avec langage de programmation associé	- SGBD : MySQL 8.0 - ORM : Eloquent (Laravel) - Langage : SQL natif + Query Builder - Migrations et Seeders Laravel - Git pour versioning	
Un logiciel de gestion de versions et de suivi de problèmes d'ordre logiciel	- GitHub pour hébergement distant - Commits réguliers avec messages descriptifs	
Une solution permettant de tester les comportements anormaux d'une application	- Validation Laravel avec règles intégrées (required, numeric, exists, image) - Middleware auth/role pour tester les accès non autorisés - Messages d'erreur utilisateur personnalisés via session flash	

2.2 Les activités de l'organisation cliente s'appuient sur au moins deux solutions applicatives opérationnelles :

Éléments	Description de l'implantation dans le centre d'examen	Remarques de la commission d'interrogation
Du code exécuté dans un navigateur Web (type client léger ou riche)	- Interface utilisateur en HTML5 via le moteur Blade - Mise en forme CSS responsive avec Bootstrap 5.3 - Interactions JavaScript côté client (dark mode toggle, validations)	

Du code exécuté sur le système d'exploitation d'une solution technique d'accès mobile	Le site est accessible sur mobile via le navigateur grâce au design responsive (Bootstrap grid, CSS media queries, viewport adaptatif)	
Du code exécuté sur le système d'exploitation d'un serveur	- Framework Laravel : logique métier en PHP 8.2 - Gestion des requêtes HTTP, accès base de données MySQL - Authentification utilisateurs, gestion des rôles et permissions	
Du code exécuté sur le système d'exploitation d'une solution technique d'accès fixe (type client lourd)	Ce projet est une application web pure. Il n'existe pas de client lourd (application desktop). L'accès se fait exclusivement via navigateur web.	

2.3 Une solution applicative peut être issue d'un développement spécifique ou de la modification du code d'un logiciel notamment open source.

2.4 Les solutions applicatives présentes dans le contexte sont opérationnelles et leur code source est accessible dans un environnement de développement opérationnel au moment de l'épreuve.